

String Matching with Finite Automata

1 Introduction

Main Idea

A powerful way to solve the string-matching problem is to build a **finite automaton** that scans the text from left to right and keeps track of how much of the pattern has matched so far.

Instead of checking the pattern separately at every shift, the automaton processes the text **one character at a time**. After reading each character, it moves to a new state that tells us:

how many characters of the pattern currently match a suffix of the text seen so far.

Key Benefit

Once the automaton is built, the text can be scanned in:

$$\Theta(n)$$

time, because each text character is examined exactly once.

The main cost lies in the **preprocessing step**, where we construct the transition function of the automaton.

2 Finite Automata: Basic Definition

A finite automaton is a mathematical machine used to process strings.

Definition of a Finite Automaton

A finite automaton M is a 5-tuple

$$M = (Q, q_0, A, \Sigma, \delta),$$

where:

- Q is a finite set of states,
- $q_0 \in Q$ is the start state,
- $A \subseteq Q$ is the set of accepting states,
- Σ is the input alphabet,
- $\delta : Q \times \Sigma \rightarrow Q$ is the transition function.

How it works

The automaton starts in state q_0 . If it is currently in state q and reads character a , then it moves to state:

$$\delta(q, a).$$

If after reading some prefix of the input, the current state belongs to A , then that prefix is accepted.

Interpretation

A finite automaton reads input from left to right and updates its state after every character. The state acts as a compact memory of what has been seen so far.

3 Final-State Function

Every automaton defines a function that tells us where the machine ends up after reading a whole string.

Final-State Function

The final-state function

$$\phi : \Sigma^* \rightarrow Q$$

is defined by:

$$\begin{aligned}\phi(\varepsilon) &= q_0, \\ \phi(wa) &= \delta(\phi(w), a), \quad w \in \Sigma^*, a \in \Sigma.\end{aligned}$$

Here:

- ε is the empty string,
- $\phi(w)$ is the state after reading the entire string w .

So a string w is accepted exactly when:

$$\phi(w) \in A.$$

4 String-Matching Automaton

Now we specialize finite automata to the string-matching problem.

Suppose the pattern is:

$$P[1 : m].$$

We want an automaton that accepts exactly those strings whose suffix is the pattern P .

Goal

After reading text prefix T_i , the automaton should know the length of the longest prefix of P that is also a suffix of T_i .

That is the core idea behind the string-matching automaton.

5 Suffix Function

To define the automaton formally, we first define an auxiliary function called the **suffix function**.

Suffix Function

For a fixed pattern P , define

$$\sigma(x) = \max\{k \mid P_k \supseteq x\},$$

where $P_k = P[1 : k]$ is the prefix of P of length k .

So:

$$\sigma(x)$$

is the length of the **longest prefix of P** that is also a **suffix of x** .

Examples

If $P = \mathbf{ab}$, then:

$$\sigma(\varepsilon) = 0,$$

$$\sigma(\mathbf{ccaca}) = 1$$

because the longest prefix of **ab** that is also a suffix of **ccaca** is **a**,

and

$$\sigma(\mathbf{ccab}) = 2$$

because **ab** itself is a suffix of **ccab**.

Important Property

If the pattern has length m , then

$$\sigma(x) = m \iff P \supseteq x.$$

So when the suffix-function value becomes m , we have matched the whole pattern.

6 Construction of the String-Matching Automaton

For a pattern $P[1 : m]$, the string-matching automaton is defined as follows.

Automaton for Pattern Matching

- State set:

$$Q = \{0, 1, 2, \dots, m\}.$$

- Start state:

$$q_0 = 0.$$

- Accepting state:

$$A = \{m\}.$$

- Transition function:

$$\delta(q, a) = \sigma(P_q a).$$

Here P_q means the first q characters of P , and a is the next input character.

Meaning of the transition function

If the automaton is in state q , then the first q characters of the pattern already match a suffix of the text processed so far.

Now a new character a arrives.

Then:

$$P_q a$$

represents the previous matched prefix extended by the new character.

We want to know the length of the longest prefix of P that is now a suffix of the new text. That is exactly:

$$\sigma(P_q a).$$

So the automaton moves to:

$$\delta(q, a) = \sigma(P_q a).$$

Interpretation of State q

State q means:

“the first q characters of the pattern match the current text suffix.”

7 Why the Transition Function Makes Sense

Suppose after reading some text prefix T_i , the automaton is in state q .

That means:

$$P_q \supseteq T_i,$$

and q is the largest such value.

Now read the next character:

$$a = T[i + 1].$$

Then the new text becomes:

$$T_i a.$$

We want the new state to represent:

the longest prefix of P that is a suffix of $T_i a$.

Because the best match before reading a was P_q , the best new match is determined by:

$$\sigma(P_q a).$$

Thus the transition rule is exactly right.

8 Two Cases of Transition

There are two important cases.

8.1 Case 1: Match Continues

If

$$a = P[q + 1],$$

then the next character matches the next pattern character.

So:

$$\delta(q, a) = q + 1.$$

That means the automaton simply moves one step forward.

8.2 Case 2: Mismatch Occurs

If

$$a \neq P[q + 1],$$

then the current partial match cannot continue directly.

The automaton must fall back to a smaller prefix of P that is still a suffix of the text read so far.

So:

$$\delta(q, a)$$

becomes some smaller value, determined by the suffix function.

Important Insight

The automaton does not start matching from scratch after a mismatch. Instead, it uses information about the pattern itself to jump to the longest possible smaller valid state.

9 Finite-Automaton Matcher Algorithm

Once the transition table δ is available, matching is very simple.

Finite-Automaton Matcher

```
FINITE-AUTOMATON-MATCHER(T,  $\delta$ , m)
1  n = T.length
2  q = 0
3  for i = 1 to n
4      q =  $\delta(q, T[i])$ 
5      if q == m
6          print "Pattern occurs with shift", i - m
```

Explanation

- Line 2 starts in state 0.
- Line 4 updates the state for the next text character.
- Line 5 checks whether the automaton has reached state m , meaning the full pattern has matched.
- If yes, the match begins at shift:

$$i - m.$$

Running Time

The automaton reads each text character once, so the matching phase runs in:

$$\Theta(n).$$

10 Core Invariant of the Matcher

The correctness of the automaton is based on a very important invariant:

Main Invariant

After reading the prefix T_i of the text,

$$\phi(T_i) = \sigma(T_i).$$

This means:

the current state of the automaton is exactly the length of the longest prefix of the pattern that is also a suffix of the text prefix read so far.

So when the current state becomes m , the full pattern has matched.

11 Lemma 1: Suffix-Function Inequality

Lemma 1 (Suffix-Function Inequality). *For any string x and character a ,*

$$\sigma(xa) \leq \sigma(x) + 1.$$

Meaning

When we append one new character to a string, the best possible matched prefix length can increase by **at most one**.

Why it is true

If $\sigma(xa) = r > 0$, then P_r is a suffix of xa .

By removing the last character from both, P_{r-1} is a suffix of x .

So:

$$r - 1 \leq \sigma(x),$$

which implies:

$$r \leq \sigma(x) + 1.$$

Hence:

$$\sigma(xa) \leq \sigma(x) + 1.$$

Interpretation

The automaton cannot jump forward by more than one state after reading one character.

12 Lemma 2: Suffix-Function Recursion

Lemma 2 (Suffix-Function Recursion). *For any string x and character a , if*

$$q = \sigma(x),$$

then

$$\sigma(xa) = \sigma(P_q a).$$

Meaning

If the longest useful suffix information about x is already captured by P_q , then to compute the new suffix value after appending a , it is enough to look at:

$$P_q a$$

instead of the entire string xa .

Why it matters

This lemma justifies the transition definition:

$$\delta(q, a) = \sigma(P_q a).$$

It tells us that the state q contains all the information needed to update the match after reading the next character.

13 The Main Correctness Theorem

Theorem 1. *If ϕ is the final-state function of the string-matching automaton for pattern P , then for every text prefix T_i ,*

$$\phi(T_i) = \sigma(T_i), \quad i = 0, 1, \dots, n.$$

Proof Idea

We prove this by induction on i .

Base Case

For $i = 0$, we have:

$$T_0 = \varepsilon.$$

So:

$$\phi(T_0) = 0 = \sigma(T_0).$$

Thus the statement is true initially.

Inductive Step

Assume:

$$\phi(T_i) = \sigma(T_i).$$

Let:

$$q = \phi(T_i), \quad a = T[i + 1].$$

Then:

$$\phi(T_{i+1}) = \phi(T_i a) = \delta(\phi(T_i), a) = \delta(q, a).$$

By the definition of δ ,

$$\delta(q, a) = \sigma(P_q a).$$

By the suffix-function recursion lemma,

$$\sigma(P_q a) = \sigma(T_i a) = \sigma(T_{i+1}).$$

Therefore:

$$\phi(T_{i+1}) = \sigma(T_{i+1}).$$

So the statement holds for $i + 1$, completing the induction.

Conclusion

The automaton always stores the correct amount of matched prefix information. Therefore, when it reaches state m , the pattern has just appeared in the text.

14 Why the Matcher is Correct

By the theorem,

$$\phi(T_i) = \sigma(T_i).$$

Now the matcher reports a match when:

$$q = m.$$

But:

$$\sigma(T_i) = m$$

means the full pattern P is a suffix of T_i .

That means the pattern ends at position i , so it starts at:

$$i - m + 1.$$

Hence the shift is:

$$i - m.$$

So the matcher prints exactly the correct shifts.

15 Computing the Transition Function

To use the automaton, we must first compute the transition table $\delta(q, a)$.

Transition-Function Computation

```
COMPUTE-TRANSITION-FUNCTION(P,  $\Sigma$ )
1  m = P.length
2  for q = 0 to m
3      for each character a in  $\Sigma$ 
4          k = min(m+1, q+2)
5          repeat
6              k = k - 1
7          until Pk is a suffix of Pq a
8           $\delta(q, a) = k$ 
9  return  $\delta$ 
```

Explanation

For every state q and character a , we want:

$$\delta(q, a) = \sigma(P_q a).$$

So we search for the largest k such that:

$$P_k \supseteq P_q a.$$

The algorithm starts with the largest possible value of k , then decreases it until the suffix condition becomes true.

What the algorithm is doing

It checks:

“If I am in state q and read character a , what is the longest prefix of the pattern that is now a suffix?”

16 Running Time of Transition Construction

Let:

$$m = |P|, \quad |\Sigma| = \text{alphabet size}.$$

The outer loops consider:

$$(m + 1)|\Sigma|$$

pairs (q, a) .

For each pair:

- the variable k may decrease up to $m + 1$ times,
- each suffix test may compare up to m characters.

So the total running time is:

$$\boxed{\mathcal{O}(m^3|\Sigma|)}.$$

Observation

This direct method is correct, but it is not very efficient. A faster method can reduce the preprocessing time to:

$$\mathcal{O}(m|\Sigma|).$$

17 Overall Complexity

The finite-automaton method has two phases.

17.1 Preprocessing

Construct the transition function:

$$\mathcal{O}(m^3|\Sigma|) \quad (\text{simple method})$$

or

$$\mathcal{O}(m|\Sigma|) \quad (\text{improved method}).$$

17.2 Matching

Scan the text:

$$\boxed{\Theta(n)}.$$

So the complete complexity is:

Overall Complexity

Preprocessing: $\mathcal{O}(m|\Sigma|)$ or $\mathcal{O}(m^3|\Sigma|)$,

Matching: $\Theta(n)$.

18 Conceptual Comparison with Earlier Methods

Method	Main Idea	Matching Time
Naive Matcher	Try every shift directly	$\mathcal{O}((n - m + 1)m)$
Rabin–Karp	Compare rolling hashes	often fast, worst case $\mathcal{O}((n - m + 1)m)$
Finite Automaton	Build state machine and scan once	$\Theta(n)$

Main Advantage

The finite-automaton matcher never backs up in the text and never rechecks earlier text characters. It always processes the text in one left-to-right pass.

19 Worked Conceptual Example

Suppose the pattern is:

$$P = \text{ababaca}.$$

Then the automaton states are:

$$0, 1, 2, 3, 4, 5, 6, 7.$$

- State 0: nothing matched yet.
- State 1: suffix matches **a**.
- State 2: suffix matches **ab**.
- ...
- State 7: full pattern **ababaca** matched.

If the automaton is in state 5, that means:

ababa

matches the current text suffix.

Now if the next character is:

c,

then the automaton goes to state 6, because:

ababac

matches.

But if the next character is not what is expected, the automaton falls back to the longest smaller prefix that can still match.

Key Understanding

The automaton is always trying to preserve the *largest possible useful partial match*.

20 Final Summary

One-Box Summary

Finite automaton $M = (Q, q_0, A, \Sigma, \delta)$ processes text one character at a time
 $Q = \{0, 1, \dots, m\}, \quad q_0 = 0, \quad A = \{m\}$
 $\sigma(x) = \max\{k \mid P_k \text{ is a suffix of } x\}$
 $\delta(q, a) = \sigma(P_q a)$
 $\phi(T_i) = \sigma(T_i)$ is the main invariant
Matcher runs in $\Theta(n)$ time after preprocessing
Direct transition construction takes $O(m^3|\Sigma|)$ time
Improved construction can be reduced to $O(m|\Sigma|)$